

Competition 1 & 2: Linear Regression From Scratch

1 Introduction

Used-car marketplaces need price estimates before a transaction is completed. A useful estimate can help a seller set expectations, help a marketplace identify unusual listings, and support consistent decisions across many combinations of vehicle age, mileage, condition, and category. Real operational data, however, is rarely clean: records may be incomplete, variables may overlap, unusual vehicles may be valid, and some columns may only become available after the event being predicted.

In this assignment, you will build a linear-regression system for a used-car marketplace. The prediction is made **after the vehicle inspection but before price negotiation and financing**. The target is `sale_price_usd`, and the final system must predict new listings whose target is unknown.

You receive a labeled public CSV but no official data dictionary or description of the synthetic data generator. Your tasks are to:

- use AI to inspect the data and develop a complete, reproducible program;
- choose and justify how to split the public data for model development;
- design preprocessing for numeric values, categories, and missing data;
- implement an unregularized linear-regression baseline with gradient descent;
- extend that implementation with L2 regularization in a separate experiment;
- choose hyperparameters and decide whether regularization is necessary using validation evidence;
- generate predictions for the instructor's unseen test CSV;
- manage the code with GitHub and understand every core block produced by AI.

The goal is not to recover the hidden data-generation formula. The goal is to build, test, explain, and manage a reliable pipeline under realistic data constraints.

2 Files released to students

The student release contains only the following files. No hidden test labels, true coefficients, anomaly labels, or data-generation code are included.

```
car_price_train.csv    # labeled development data
T1&2.pdf              # authoritative project handout
```

3 Your workflow

Complete Part I before adding any regularization. Then preserve the Part I result as a baseline while conducting the Part II experiment. Unless stated otherwise below, both parts follow the same package restrictions and use RMSE as the only prediction metric.

Part I: Linear regression without regularization

1. Inspect and reason

Use Python and AI to inspect shapes, types, missingness, ranges, category levels, and relationships among columns. Identify the ID and target. For every proposed predictor, ask whether it would exist at the stated prediction time. You do not need to discover the hidden data-generation formula.

2. Choose your own validation design

You decide how to divide the public CSV for development. A single holdout split, repeated splits, or cross-validation are all possible. Explain why your design is appropriate. All learned preprocessing statistics must be fitted without using the corresponding validation rows.

3. Ask AI to build a reusable preprocessing pipeline

Your program must handle numeric missing values, categorical missing values, scaling, and categorical encoding. It must produce the same feature columns in the same order for training, validation, and final test rows. The final test may have more missing values and category levels not observed in the public CSV.

4. Ask AI to implement and test gradient descent

At minimum, choose and investigate:

- learning rate;
- maximum number of iterations;
- a numerical convergence rule or stopping tolerance.

Check that losses and predictions remain finite and that the training loss behaves sensibly. You may ask AI to design small tests, but you must understand what each test demonstrates.

5. Select an unregularized baseline using validation RMSE

Use only RMSE for model selection:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}.$$

Decide which features, preprocessing rules, and hyperparameters to keep. Save the configuration and validation RMSE of this unregularized baseline before beginning Part II.

Part II: Explore the necessity of regularization

6. Extend the model with L2 regularization

Add an L2 strength $\lambda \geq 0$ to the same gradient-descent implementation:

$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^p w_j^2.$$

Do not penalize the intercept b . Confirm that setting $\lambda = 0$ reproduces the Part I objective and gradient. Explain how the penalty changes the coefficient gradient and why feature scaling matters when coefficients are penalized.

7. Run a controlled comparison

Identify at least one strongly correlated pair of predictors. Compare $\lambda = 0$ with at least three positive L2 strengths while holding the data split, feature set, preprocessing, initialization, and other training settings fixed. Record validation RMSE and the L2 norm of the fitted coefficient vector for every value.

Then compare $\lambda = 0$ with your selected positive value across at least three different split seeds or validation folds. Report the mean and range of RMSE. This second comparison helps distinguish a repeatable benefit from a lucky split.

8. Decide whether regularization is necessary

Use your validation evidence to choose either the unregularized Part I pipeline or a regularized Part II pipeline for the classroom test. A positive λ is not automatically better, and choosing $\lambda = 0$ is acceptable when the evidence supports it. Discuss predictive RMSE, coefficient shrinkage, and the dataset's correlated predictors. Once the decision is made, refit only the selected pipeline using all appropriate public rows.

4 Technical rules

4.1 Allowed tools

You may use Python's standard library, NumPy, Pandas, and Matplotlib. AI may generate all parts of your program, including preprocessing, model training, tuning, tests, and file handling.

4.2 Not allowed

- Machine-learning or statistical-modeling packages, including scikit-learn, statsmodels, XGBoost, LightGBM, CatBoost, TensorFlow, and PyTorch.
- A ready-made regression model or optimization routine.
- A direct least-squares solver, matrix inverse, pseudoinverse, polynomial-fit helper, or equivalent short-cut for fitting the model.

Array operations such as matrix multiplication, means, standard deviations, and elementwise arithmetic are allowed.

Part I must use ordinary mean squared error with $\lambda = 0$. Part II must implement the L2 penalty shown above. L1 or other penalties may be explored only as optional extensions and may not replace the required L2 comparison. Regularization does not make an unavailable or leaked feature valid.

4.3 Minimum model interface

Your model must support the following two operations. Its internal organization is your choice.

```
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

The fitted parameters must be learned through gradient-descent updates. The same implementation should support $\lambda = 0$ for Part I and $\lambda > 0$ for Part II.

5 Classroom hidden test

During the final test, the instructor will provide a CSV containing 600 new feature rows and no target. Make the train and test file locations easy to change near the top of your program. Replace the test path, run your program, and write `predictions.csv` with exactly these columns:

```
listing_id,sale_price_usd_pred
```

There must be exactly one finite prediction for every test ID. The instructor will join predictions to labels by `listing_id`; row order is not used. The hidden features are provided once, and the hidden score is not available for further tuning.

6 GitHub repository requirement

Use GitHub to manage the code throughout the project. Create one repository for your submission and use Git commits to record meaningful stages such as data inspection, preprocessing, model implementation, validation, and final prediction preparation. Do not wait until the end and upload the entire project as a single commit. The repository may be public or private according to the instructor's course policy.

The repository must contain a top-level `README.md` that explains:

- the purpose of the project and its package restrictions;
- the required Python version and exact dependency-installation command;
- the command or steps used to train the model;
- the command or steps used to generate predictions for a new test CSV;
- where the train and test CSV paths are configured;
- the required schema and location of `predictions.csv`.

Include a `.gitignore` that excludes virtual environments, caches, temporary files, and instructor-provided hidden data. Never commit hidden test features or labels to GitHub.

7 What to submit

Please refer to our first slides for competition.

8 What good AI collaboration looks like

A useful AI prompt gives the prediction time, package restrictions, current data shapes, and the specific component you want help with. A useful verification step produces evidence: an assertion, a tiny controlled example, a shape check, a loss trace, or a comparison of validation RMSE. “The AI said it was correct” is not evidence. If an AI suggestion violates a rule or makes an unjustified assumption, record that decision in your log and ask for a narrower revision.

A strong submission is not the one with the longest code or the most elaborate prompt. It is the one whose author can justify, test, and explain every important decision.